

Pairs Trading Performance Under Reinforcement Learning

Yuan-Chen Ma
Department of Mathematics
National Central University

Abstract—This report investigates a pairs trading framework that combines cointegration-based statistical screening, hierarchical clustering, and Deep Q-Network (DQN) reinforcement learning for dynamic pair selection. The proposed framework first constructs a candidate stock universe and evaluates pairwise residual stationarity through an Engle–Granger-style procedure. A Cross-Cointegration Voting (CCA-V) mechanism is then employed to identify stocks that repeatedly appear in relatively stable clustering configurations. The selected stocks form a candidate set of 666 trading pairs, whose recent spread behavior is represented by rolling Z-Scores and fixed-dimensional edge-padded state vectors. A DQN subsequently learns to select one candidate pair at each monthly decision point. The final implementation uses a walk-forward evaluation in which a trading decision is made before the reward for the subsequent month is observed, while the hedge-ratio parameters remain fixed at estimates obtained from the training period. Results over 2020–2025 produce positive cumulative returns but relatively modest risk-adjusted performance and substantial drawdown. The analysis therefore emphasizes not only absolute return, but also the importance of market exposure, concentration risk, statistical stability, and realistic backtesting assumptions when evaluating a market-neutral trading framework.

I. INTRODUCTION

Pairs trading is a statistical arbitrage strategy designed to exploit relative mispricing between two assets. The basic idea is to form a long–short position in two securities whose prices exhibit a stable long-run relationship. When the relative spread deviates substantially from its historical equilibrium, the strategy takes an opposing position with the expectation that the spread will subsequently revert.

It is important to distinguish statistical correlation from cointegration in this context. Correlation measures the degree of linear association between two random variables over a given sample, whereas cointegration concerns the existence of a stationary linear combination of non-stationary time series. Thus, two stocks may exhibit high correlation over a particular period without possessing the long-run equilibrium relationship required by a cointegration-based pairs trading strategy.

Although conventional pairs trading can identify individual pairs with mean-reverting behavior, selecting a suitable pair in a changing market environment remains difficult. Recent studies have increasingly combined deep learning and reinforcement learning with pairs trading. Xia et al. proposed a hierarchical deep learning framework

that uses graph-based attention mechanisms for pair selection and deep reinforcement learning for trading execution [2]. Xu and Luo proposed a two-level reinforcement learning framework for pairs trading [3], while Lai investigated the combination of cointegration-based clustering and reinforcement learning for pair selection [1]. These studies motivate the use of learning-based methods for dynamic pair selection, while the present study focuses on combining residual-stationarity-based clustering, CCA-V voting, and a DQN agent for discrete monthly pair selection.

The main research questions considered in this study are:

- 1) Can residual-stationarity information be used to construct a more stable candidate stock pool through the CCA-V clustering and voting procedure?
- 2) Can a Deep Q-Network learn to select among a large number of candidate pairs using recent Z-Score trajectories as the state?
- 3) Does the DQN-based pair-selection mechanism provide useful performance relative to the underlying pairs trading framework?
- 4) Under which market conditions does the proposed strategy perform well or poorly, and what methodological limitations may contribute to the observed performance?

The complete framework is divided into two major components. The first component constructs the candidate pair pool through data filtering, residual stationarity analysis, hierarchical clustering, and CCA-V voting. The second component formulates the pair-selection problem as a Markov Decision Process (MDP) and trains a DQN agent to select one candidate pair at each monthly decision point.

As illustrated in Figure 1, the pair-pool formation pipeline follows a sequential procedure. Raw price data are first filtered to obtain the candidate stock universe. Pairwise residual-stationarity tests are then performed, and the resulting p-values are aggregated to form the dissimilarity matrices used for hierarchical clustering. Multiple clustering configurations are evaluated by clustering-quality metrics, and the surviving configurations participate in the CCA-V voting mechanism. Finally, the

highest-voted stocks are retained and combined pairwise to form the trading universe.

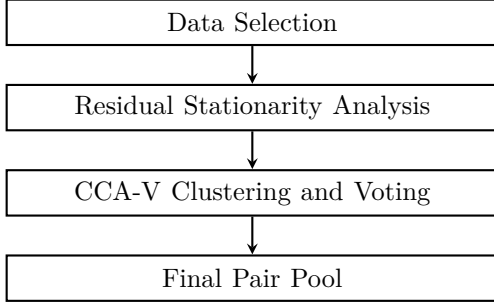


Fig. 1: Sequential Pipeline of Pair Pool Formation

Once the pair pool has been established, the pair-selection problem is formulated as a sequential decision process. At a monthly decision point, the agent observes the current state, selects one candidate pair, and subsequently receives the realized return from the following trading month. This temporal structure is important for avoiding the use of future realized rewards when making the current decision.

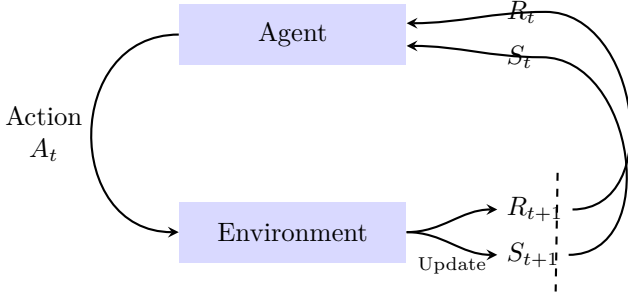


Fig. 2: Reinforcement Learning Agent–Environment Interaction

II. PAIR POOL FORMATION AND FEATURE ENGINEERING

A. Data Selection and Preprocessing

The implementation begins with historical stock-price data for the training period 2010–2019 and the testing period 2020–2025. The dataset is first restricted to common stocks using the available issue-type indicator. Daily observations are then organized into a date-by-ticker price matrix using closing prices.

Missing observations are handled by forward filling for at most five consecutive observations. Stocks with more than 5% missing values after this procedure are removed from the price matrix. The average trading volume over the training period is then calculated for the remaining securities, and the 100 stocks with the largest

average trading volume are selected as the initial candidate universe.

The implementation subsequently retains only stocks appearing in both the training and test matrices in order to maintain a consistent universe across the two periods. This choice simplifies the experimental implementation, but it also introduces a potential survivorship or future-availability selection bias because test-period availability contributes to the final common-ticker set. A fully historical backtest would instead reconstruct the investable universe using only information available at each historical decision date.

The data are divided chronologically into

$$\mathcal{D}_{\text{train}} = \{2010, \dots, 2019\}$$

and

$$\mathcal{D}_{\text{test}} = \{2020, \dots, 2025\}.$$

The training period is used to construct the statistical relationships, the initial DQN, and the hedge-ratio parameters used in the test period.

B. Correlation versus Cointegration

Correlation and cointegration measure fundamentally different properties of time series. Let X_t and Y_t denote two asset price processes. A correlation coefficient describes the degree of linear association between the two variables over a particular sample period. It does not imply that a stable long-run equilibrium exists.

By contrast, suppose that

$$X_t \sim I(1), \quad Y_t \sim I(1).$$

If there exists a non-zero vector β such that

$$\beta_1 X_t + \beta_2 Y_t \sim I(0),$$

then X_t and Y_t are said to be cointegrated. In a pairs trading context, this stationary linear combination can be interpreted as a spread that may exhibit mean-reverting behavior.

Therefore, a highly correlated pair is not necessarily a suitable cointegrated pair. The objective of the statistical screening in this study is to identify relationships for which the residual spread has evidence of stationarity.

C. Residual Stationarity Analysis

For each candidate stock pair, the implementation divides the training observations into calendar-month groups. Within each month containing more than 15 observations, an OLS regression is estimated:

$$Y_t = \beta_0 + \beta_1 X_t + \epsilon_t.$$

The corresponding residual is

$$\hat{\epsilon}_t = Y_t - \hat{\beta}_0 - \hat{\beta}_1 X_t.$$

The residual is then examined using an Augmented Dickey–Fuller (ADF)-type regression without a deterministic constant:

$$\Delta \hat{\epsilon}_t = \omega \hat{\epsilon}_{t-1} + \sum_{k=1}^p \phi_k \Delta \hat{\epsilon}_{t-k} + u_t.$$

The associated hypotheses are

$$H_0 : \omega = 0,$$

corresponding to a unit root in the residual, and

$$H_1 : \omega < 0,$$

corresponding to residual stationarity.

The test statistic is

$$\tau = \frac{\hat{\omega}}{\text{SE}(\hat{\omega})}.$$

A smaller p-value provides stronger evidence against the null hypothesis of residual non-stationarity. Importantly, the p-value should not be interpreted as a measure of whether two stocks “belong to the same cluster.” Instead, it provides statistical evidence concerning residual stationarity, and the resulting collection of p-values is subsequently used as dissimilarity information in the clustering procedure.

The implementation should therefore be viewed as an Engle–Granger-style residual stationarity screening procedure. Since the first-stage regression parameters are estimated from the data and the ADF test is applied to the resulting residuals, the associated cointegration inference is not identical to applying a generic standard ADF test to an exogenous time series.

Moreover, because the implementation performs the procedure within monthly groups, each individual test uses a relatively short sample. Thus, the monthly p-values are treated primarily as a longitudinal screening signal for the subsequent CCA-V procedure rather than as independent definitive proofs of a stable long-run cointegration relationship.

D. CCA-V Clustering and Voting System

For each stock pair (i, j) , the monthly ADF p-values are stored as

$$\mathcal{P}_{ij} = \left\{ p_{ij}^{(1)}, p_{ij}^{(2)}, \dots, p_{ij}^{(T)} \right\}.$$

For a selected percentile level q , the pairwise dissimilarity matrix is defined by

$$M_q(i, j) = \text{Percentile}_q(\mathcal{P}_{ij}).$$

Since smaller p-values correspond to stronger evidence against residual non-stationarity, smaller entries of M_q represent stronger pairwise statistical evidence of residual stationarity.

The study considers

$$q \in \{0.10, 0.12, 0.14, 0.16, 0.18, 0.20\}$$

and

$$K \in \{2, 3, 4, 5, 6\}.$$

Therefore, a total of

$$6 \times 5 = 30$$

clustering configurations are evaluated.

For each q , complete-linkage hierarchical agglomerative clustering is applied to the dissimilarity matrix. Two complementary clustering-quality criteria are then evaluated.

For an observation i , the Silhouette Score is

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}},$$

where

$$a(i) = \frac{1}{|C_i| - 1} \sum_{\substack{j \in C_i \\ j \neq i}} d(i, j)$$

is the average dissimilarity between i and the other members of its cluster, and

$$b(i) = \min_{C \neq C_i} \frac{1}{|C|} \sum_{j \in C} d(i, j)$$

is the smallest average dissimilarity between i and another cluster. Larger Silhouette values indicate stronger separation.

In the implementation, the Silhouette Score uses M_q directly as a precomputed dissimilarity matrix. The Davies–Bouldin Score, however, is evaluated from the row-wise representation of M_q . Specifically, stock i is represented by

$$\mathbf{m}_i = [M_q(i, 1), M_q(i, 2), \dots, M_q(i, N)],$$

and the resulting feature vectors are supplied to the Davies–Bouldin calculation. The general Davies–Bouldin form is

$$DB = \frac{1}{K} \sum_{i=1}^K \max_{j \neq i} \left(\frac{S_i + S_j}{M_{ij}} \right),$$

where S_i denotes the dispersion of cluster i , and M_{ij} represents the separation between the corresponding cluster representations. Lower values indicate better clustering according to this criterion.

The implementation uses the median Silhouette Score and median Davies–Bouldin Score across the 30 configurations as screening thresholds. A clustering configuration is admitted to the voting stage if it satisfies at least one of the conditions

$$s_{\text{config}} \geq \text{median}(s),$$

or

$$DB_{\text{config}} \leq \text{median}(DB).$$

Thus, the implemented screening rule is an “OR” condition rather than an “AND” condition.

For every surviving clustering configuration, each cluster containing at least two stocks is examined. The cluster with the smallest mean within-cluster dissimilarity is selected as the most stable cluster for that configuration. Every stock contained in this cluster receives one vote. The final stock stability score is therefore the number of votes received across the surviving clustering configurations.

The 37 stocks with the highest vote frequencies are retained as the final stock pool.

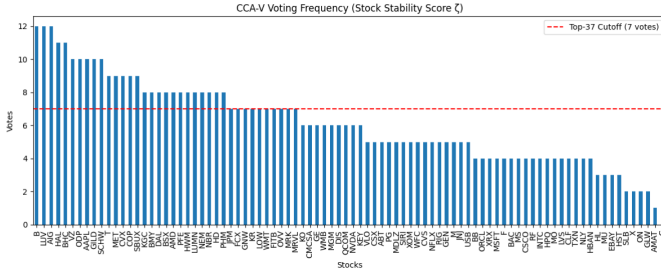


Fig. 3: CCA-V Voting Frequency (Stock Stability Score)

E. Rolling Z-Score and Edge Padding

After CCA-V screening, the final 37 stocks generate

$$\binom{37}{2} = \frac{37 \times 36}{2} = 666$$

candidate trading pairs.

For each pair, the training-period OLS regression provides the estimated coefficients

$$\hat{\beta}_0, \quad \hat{\beta}_1.$$

The raw residual is

$$\hat{\epsilon}_t = Y_t - \hat{\beta}_0 - \hat{\beta}_1 X_t.$$

A 20-day rolling mean and standard deviation are then used to standardize the residual:

$$\mu_{20,t} = \frac{1}{20} \sum_{k=0}^{19} \hat{\epsilon}_{t-k},$$

$$\sigma_{20,t} = \text{SD}(\hat{\epsilon}_{t-19}, \dots, \hat{\epsilon}_t),$$

and

$$Z_t = \frac{\hat{\epsilon}_t - \mu_{20,t}}{\sigma_{20,t}}.$$

The implementation allows fewer than 20 observations at the beginning of a sequence and subsequently fills invalid standard deviations and rolling means to maintain numerical stability. These initial values should therefore be interpreted as implementation-dependent warm-up values rather than fully populated 20-day statistics.

Because the number of trading days varies from month to month, monthly Z-Score sequences have different lengths. Let N_m be the number of available trading observations in month m , and define

$$N_{\max} = \max_m N_m.$$

To create a fixed-dimensional input for the neural network, the implementation uses edge padding:

$$\tilde{Z}_{m,t} = \begin{cases} Z_{m,t}, & 1 \leq t \leq N_m, \\ Z_{m,N_m}, & N_m < t \leq N_{\max}. \end{cases}$$

Hence, each month is represented using a matrix of dimension

$$666 \times N_{\max}.$$

The padded observations are artificial copies of the final observed value and are used solely for dimensional compatibility. They are not treated as additional market observations.

After transposition and flattening, each month becomes one state vector for the DQN.

III. TRADING STRATEGY AND RL FORMULATION

A. Trading Execution Logic

The trading strategy uses the Z-Score series as a standardized spread measure. A five-day moving average and standard deviation are computed from the Z-Score:

$$\mu_{5,t} = \text{MA}_5(Z_t),$$

$$\sigma_{5,t} = \text{SD}_5(Z_t).$$

The entry conditions are evaluated using information from the previous trading day. For a short-spread position, the strategy enters when

$$Z_{t-1} > \mu_{5,t-1} + \sigma_{5,t-1}.$$

The corresponding position is long X and short Y . For a long-spread position, the strategy enters when

$$Z_{t-1} < \mu_{5,t-1} - \sigma_{5,t-1},$$

which corresponds to short X and long Y .

Positions are exited when the previous day’s Z-Score has crossed the five-day moving average in the appropriate direction. A position is also forcibly closed at the end of a month.

The entry and exit rules are therefore deterministic conditional on the selected pair. The reinforcement learning component does not directly learn the entry threshold; instead, it learns which candidate pair should be selected for the next trading period.

B. Hedge-Ratio-Based Entry Weights

The hedge ratio is obtained from the OLS regression

$$Y_t = \hat{\beta}_0 + \hat{\beta}_1 X_t + \hat{\epsilon}_t.$$

At the entry date t_e , the implementation calculates the notional weights immediately before execution as

$$w_{1,e} = \frac{\hat{\beta}_1 X_{t_e-1}}{\hat{\beta}_1 X_{t_e-1} + Y_{t_e-1}},$$

$$w_{2,e} = \frac{Y_{t_e-1}}{\hat{\beta}_1 X_{t_e-1} + Y_{t_e-1}},$$

so that, whenever the denominator is well-defined,

$$w_{1,e} + w_{2,e} = 1.$$

Importantly, these weights are fixed at entry for the duration of the trade. They are not recomputed daily during an open position.

For a short-spread position, namely long X and short Y , the implemented daily return contribution is

$$r_t^{(A)} = w_{1,e} \frac{X_t - X_{t-1}}{X_{t_e}} + w_{2,e} \frac{Y_{t-1} - Y_t}{Y_{t_e}}.$$

For a long-spread position, namely short X and long Y ,

$$r_t^{(B)} = w_{1,e} \frac{X_{t-1} - X_t}{X_{t_e}} + w_{2,e} \frac{Y_t - Y_{t-1}}{Y_{t_e}}.$$

The monthly reward associated with a pair is obtained by summing its daily return contributions over the month:

$$R_m = \sum_{t \in \mathcal{T}_m} r_t.$$

No explicit total portfolio capital limit is imposed in this calculation. Instead, the return is represented as a normalized long-short return generated by the hedge-ratio-based notional weights.

C. MDP Formulation

The pair-selection problem is formulated as a finite-horizon Markov Decision Process

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma).$$

At monthly decision point t , the state S_t contains the Z-Score trajectories of all 666 candidate pairs during month t . The state is therefore represented as

$$S_t \in \mathbb{R}^{666 \times N_{\max}},$$

before flattening for neural-network input.

The action space is discrete:

$$\mathcal{A} = \{1, 2, \dots, 666\}.$$

Action

$$A_t = i$$

means that the i -th candidate pair is selected for the subsequent trading month.

The reward is not the return of the current state month. Instead, the implementation uses the reward of the subsequent month:

$$S_t \longrightarrow A_t \longrightarrow R_{t+1} \longrightarrow S_{t+1}.$$

This temporal separation is important because it prevents the subsequent realized return from being used to determine the action that generated that same return.

The reward associated with action $A_t = i$ is

$$R_{t+1} = \sum_{\tau \in \mathcal{T}_{t+1}} r_{i,\tau}.$$

Thus, the DQN learns to associate the observed behavior of candidate pairs with their future trading performance.

D. Deep Q-Network

A tabular Q-learning approach would require a separate value estimate for each state-action pair. Because the state is high-dimensional and contains the trajectories of 666 candidate pairs, the state space is far too large for such a direct tabular representation.

The DQN instead approximates the action-value function using a neural network:

$$Q(S_t, a; \theta) \approx Q^*(S_t, a),$$

where θ denotes the trainable network parameters. The selected action during exploitation is

$$A_t = \arg \max_{a \in \mathcal{A}} Q(S_t, a; \theta).$$

The advantage of this formulation is that a single neural network produces the estimated value of all 666 discrete actions for a given state.

E. Epsilon-Greedy Exploration

During training, the implementation uses an epsilon-greedy strategy. With probability ϵ_k , a random action is selected; otherwise, the action with the largest estimated Q-value is selected:

$$A_k = \begin{cases} \text{random action,} & \text{with probability } \epsilon_k, \\ \arg \max_{a \in \mathcal{A}} Q(S_k, a; \theta), & \text{with probability } 1 - \epsilon_k. \end{cases}$$

The exploration probability is initialized at

$$\epsilon_0 = 1$$

and decays according to

$$\epsilon_k = 0.995^k.$$

Therefore, the policy becomes progressively more exploitative during training. This geometric schedule does not by itself provide a formal guarantee of convergence of the nonlinear DQN; rather, it provides a practical mechanism for balancing exploration and exploitation.

F. Bellman Equation and Target Network

For a discounted MDP, the optimal action-value function satisfies the Bellman optimality equation

$$Q^*(s, a) = \mathbb{E} \left[R_{t+1} + \gamma \max_{a' \in \mathcal{A}} Q^*(S_{t+1}, a') \mid S_t = s, A_t = a \right].$$

In the DQN implementation, the temporal-difference target is approximated by

$$Y_t = R_{t+1} + \gamma(1 - D_t) \max_{a' \in \mathcal{A}} Q_{\text{target}}(S_{t+1}, a'; \theta^-),$$

where D_t indicates whether the transition terminates and θ^- denotes the target-network parameters.

The discount factor is

$$\gamma = 0.5.$$

The implementation maintains two networks:

$$Q_{\text{policy}}(S, a; \theta)$$

and

$$Q_{\text{target}}(S, a; \theta^-).$$

The target network is periodically synchronized with the policy network rather than being modified after every gradient update. During initial training, synchronization occurs every

$$C = 10$$

gradient-update steps.

The purpose of the target network is to reduce the rapid movement of the regression target and thereby improve numerical training stability. For $0 \leq \gamma < 1$, the Bellman optimality operator satisfies the contraction property

$$\|TQ_1 - TQ_2\|_\infty \leq \gamma \|Q_1 - Q_2\|_\infty$$

under the standard discounted MDP setting. This property concerns the Bellman operator itself and should not be interpreted as a guarantee that nonlinear gradient-based DQN training will always converge or that Q-values cannot become unstable in practical financial applications.

G. Experience Replay and Loss Function

The implementation stores historical transitions in a replay buffer with capacity 1000.

A mini-batch of size $B = 32$ is sampled uniformly from the buffer.

The DQN objective is the mean squared temporal-difference error:

$$\mathcal{L}(\theta) = \frac{1}{B} \sum_{i=1}^B (Y_i - Q_{\text{policy}}(S_i, A_i; \theta))^2.$$

The gradient of this objective is

$$\nabla_{\theta} \mathcal{L}(\theta) = -\frac{2}{B} \sum_{i=1}^B (Y_i - Q_{\text{policy}}(S_i, A_i; \theta)) \nabla_{\theta} Q_{\text{policy}}(S_i, A_i; \theta).$$

The gradients are optimized using the Adam optimizer.

IV. IMPLEMENTATION AND RESULTS

A. DQN Architecture and Training Configuration

The implemented DQN consists of a fully connected network with four hidden layers:

$$1024 \rightarrow 512 \rightarrow 128 \rightarrow 512.$$

Each hidden layer uses the ReLU activation function,

$$\text{ReLU}(x) = \max(0, x).$$

The final output layer has 666 neurons, corresponding to the 666 candidate pair-selection actions.

The training optimizer is Adam with learning rate

$$\eta = 0.001.$$

The replay-buffer capacity is 1000 transitions, the mini-batch size is 32, and the discount factor is 0.5.

Training is performed for at most 300 episodes. Each episode repeatedly traverses the same chronological sequence of historical monthly transitions. Therefore, the 300 episodes should be understood as repeated optimization passes over the in-sample historical sequence, rather than as 300 independent market environments.

B. Early Stopping

The implementation records the total reward obtained in each training episode:

$$R^{(\text{episode})} = \sum_t R_{t+1}.$$

The best episode reward observed so far is tracked. If the episode reward fails to exceed the historical best for 100 consecutive episodes, training is stopped.

This procedure can be regarded as an empirical regularization mechanism. However, it does not constitute a formal proof that the model is free from overfitting, particularly because the stopping criterion is based on the training reward rather than on an independent validation set.

C. Walk-Forward Out-of-Sample Evaluation

The test period uses the hedge-ratio coefficients estimated from the training period. For each pair, $(\hat{\beta}_0, \hat{\beta}_1)$ are obtained from the training data and carried into the test period. No OLS re-estimation based on future test observations is performed.

The DQN itself is evaluated using a sequential walk-forward procedure. At monthly decision point t , the model first observes S_t and selects

$$A_t = \arg \max_{a \in \mathcal{A}} Q(S_t, a; \theta_t).$$

The reward for the subsequent month is then realized and stored in the replay buffer:

$$S_t \rightarrow A_t \rightarrow R_{t+1}.$$

Only after this outcome has been observed is it used for further DQN updates. The model is therefore allowed to adapt to previously observed test-period outcomes, but not to use future realized rewards to retroactively determine earlier trading actions.

This distinction is important. The evaluation is therefore more accurately described as an adaptive walk-forward out-of-sample experiment rather than a single fixed-model out-of-sample test.

During the test period, exploration is disabled:

$$\epsilon = 0,$$

so that the action is chosen deterministically from the maximum estimated Q-value.

The target network is synchronized after each monthly test update in the final backtesting procedure. Specifically, during this adaptive walk-forward phase, the model performs 5 mini-batch gradient updates from the replay buffer at each monthly step before synchronizing the target network, allowing the agent to continuously adapt to the evolving market regime.

D. Performance Measures

Let r_t denote the monthly strategy return. The cumulative return is computed as

$$CR_T = \prod_{t=1}^T (1 + r_t) - 1.$$

The mean monthly return is

$$\bar{r} = \frac{1}{T} \sum_{t=1}^T r_t.$$

The annualized volatility used in the implementation is

$$\sigma_{\text{ann}} = \sqrt{12} \text{SD}(r_t).$$

The implemented Sharpe ratio uses a zero risk-free rate:

$$SR = \frac{12\bar{r}}{\sqrt{12} \text{SD}(r_t)}.$$

Thus, the reported Sharpe ratio is an annualized Sharpe ratio under the assumption

$$R_f = 0.$$

This is an implementation choice rather than a statement that the actual risk-free rate is zero.

For the downside measure, the implementation considers only negative monthly returns. Let

$$\mathcal{N} = \{r_t : r_t < 0\}.$$

The downside dispersion used is

$$\sigma_{\text{down,ann}} = \sqrt{12} \text{SD}(\mathcal{N}),$$

and the corresponding ratio is

$$So = \frac{12\bar{r}}{\sigma_{\text{down,ann}}}.$$

Therefore, the implementation should be interpreted as a negative-return-based downside measure rather than a universal definition of the Sortino ratio.

Let V_t denote the cumulative wealth process,

$$V_t = \prod_{i=1}^t (1 + r_i).$$

The running maximum is

$$V_t^{\max} = \max_{1 \leq i \leq t} V_i,$$

and the drawdown is

$$DD_t = \frac{V_t - V_t^{\max}}{V_t^{\max}}.$$

The maximum drawdown is

$$MDD = \min_t DD_t.$$

Finally, the implementation computes

$$Calmar_{\text{impl}} = \frac{12\bar{r}}{|MDD|}.$$

This quantity should be distinguished from the conventional CAGR-based Calmar ratio because its numerator is the annualized mean monthly return rather than CAGR.

Evaluation Metrics	Test 1	Test 2
Cumulative Returns	26.95%	26.16%
Monthly Returns	0.53%	0.54%
Sharpe Ratio	0.3138	0.2994
Sortino Ratio	0.4289	0.5056
Maximum Drawdown	-33.55%	-36.29%
Calmar Ratio	0.1882	0.1792

TABLE I: Reported Out-of-Sample Performance

E. Out-of-Sample Results

The reported out-of-sample performance is summarized in Table I.

Note: The reported Sharpe ratios are calculated under the simplifying assumption $R_f = 0$. Since the contemporaneous short-term U.S. Treasury bill rate was not incorporated into the current implementation, these values may overstate risk-adjusted performance relative to a Sharpe ratio based on the actual risk-free rate.

The reported results show that the strategy generates positive cumulative returns over the evaluation period. However, the risk-adjusted performance is relatively modest, with the reported Sharpe ratios remaining close to 0.3. These Sharpe ratios are calculated under the simplifying assumption $R_f = 0$, rather than using the contemporaneous risk-free rate. In addition, the maximum drawdown is larger than 33% in magnitude.

These results imply that positive cumulative return alone is not sufficient to establish the effectiveness of the proposed strategy. The substantial drawdown indicates that the strategy is exposed to meaningful loss episodes, while the modest Sharpe ratio indicates that the observed returns are relatively small compared with the variability of the monthly returns. A risk-adjusted evaluation using the contemporaneous risk-free rate may therefore lead to a different assessment of the strategy’s performance.

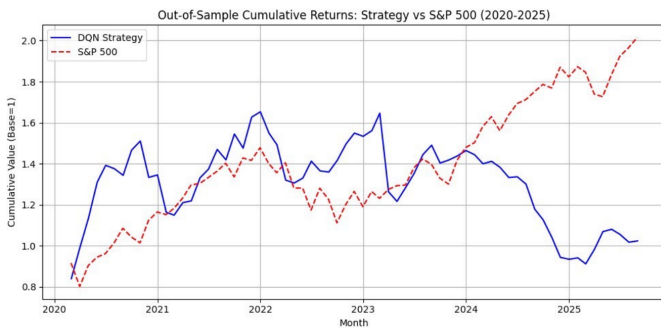


Fig. 4: Out-of-Sample Cumulative Returns: Strategy vs S&P 500

F. Market-Regime and Performance Discussion

The full-period strategy performance is below that of a long-only S&P 500 benchmark. This comparison is useful, but it should be interpreted carefully. Pairs trading is designed to reduce directional market exposure through

long-short positions and therefore does not have the same objective or risk exposure as a long-only market investment.

Consequently,

$$R_{\text{pairs}} < R_{\text{S\&P500}}$$

does not by itself establish that the strategy is unsuccessful. However, the result does indicate that the strategy does not provide sufficient risk-adjusted performance to dominate a simple long-only benchmark over the full evaluation horizon.

A more informative comparison should consider the strategy’s market beta, return volatility, drawdown, Sharpe ratio, and correlation with market returns. For example, if $R_{s,t}$ denotes the strategy return and $R_{m,t}$ denotes the market return, the market beta can be estimated by

$$\hat{\beta}_{s,m} = \frac{\text{Cov}(R_s, R_m)}{\text{Var}(R_m)}.$$

Similarly, the return correlation is

$$\rho_{s,m} = \frac{\text{Cov}(R_s, R_m)}{\sigma_s \sigma_m}.$$

These quantities provide a more direct assessment of whether the strategy actually exhibits low directional market exposure.

The observed performance deterioration in later market periods may also be related to changes in the stability of historical stock relationships. In particular, a pair that exhibits residual stationarity during the training period may subsequently experience structural changes in its relative-price dynamics. Rather than asserting that a specific market event necessarily caused the deterioration, this study treats the change in market conditions as a hypothesis to be investigated.

Useful diagnostics for this hypothesis include the number of statistically stable pairs over time, the distribution of the cointegration-related p-values, cluster stability, spread persistence, and the frequency with which the DQN selects the same stocks or pairs.

The current single-pair action space can also create concentration risk. Because the DQN must select one pair from the 666 candidates at each decision point, the resulting portfolio can become concentrated in a small number of securities or sectors. This concentration may be particularly important during structural market changes.

G. Methodological Limitations

Several limitations should be considered when interpreting the results.

First, the final common-ticker universe is constructed using stocks that are present in both the training and test datasets. This improves implementation consistency but can introduce survivorship or future-availability selection

bias. A more rigorous study should use historical index membership and security availability at each decision date.

Second, the implementation uses closing-price data and does not explicitly perform corporate-action adjustments within the code. Therefore, the treatment of stock splits and dividends depends on the properties of the supplied dataset. This should be verified before interpreting the backtest as a fully economically realistic trading simulation.

Third, missing observations are forward-filled for up to five observations. Although this prevents small gaps from interrupting the time series, forward filling may produce stale-price periods when the underlying missingness has an economic interpretation.

Fourth, the monthly residual stationarity tests use relatively short samples. This is a practical design choice for the CCA-V screening mechanism but limits the strength of long-run cointegration inference.

Fifth, no fixed random seed is specified in the final implementation. Consequently, network initialization, epsilon-greedy exploration, and replay-buffer sampling may produce variation across independent runs. Reproducibility would be improved by controlling the relevant random number generators and reporting results over multiple independent seeds.

Sixth, the present state representation contains recent Z-Score trajectories but does not explicitly include other market variables such as volatility, volume, sector information, market returns, or interest rates. Such additional features may help the agent distinguish different market regimes.

Finally, the trading model does not yet include transaction fees, slippage, or short-borrowing costs. The reported performance should therefore be interpreted as a simplified backtest rather than a direct estimate of implementable trading profitability.

Additionally, a critical methodological limitation regarding the simulated trading environment must be noted. The implementation employs a defensive programming mechanism that clips the maximum monthly loss at -99% to prevent mathematical inversion during cumulative return calculations. In a realistic leveraged trading environment, a loss approaching this magnitude would trigger a margin call or result in complete capital depletion. Thus, while the continuous compounding framework maintains numerical stability, the reported drawdowns emphasize the necessity of implementing strict stop-loss boundaries and risk-management protocols in future deployments.

V. FUTURE WORK AND SYSTEM EXTENSIONS

The observed results motivate several directions for further research.

A. Realistic Trading Costs and Risk Adjustment

Future experiments should explicitly model transaction costs:

$$TC_t = TC_t^{\text{fee}} + TC_t^{\text{slippage}} + TC_t^{\text{borrow}}.$$

The resulting net return would be

$$r_t^{\text{net}} = r_t^{\text{gross}} - TC_t.$$

In addition, the risk-free rate should be incorporated into the Sharpe ratio using a contemporaneous short-term U.S. Treasury bill rate:

$$SR = \sqrt{K} \frac{\mathbb{E}[R_t - R_{f,t}]}{\text{SD}(R_t - R_{f,t})},$$

where K depends on the return frequency.

B. Market-Exposure Analysis

The strategy should be evaluated using

$$\beta_{s,m}, \quad \rho_{s,m}, \quad \sigma_s, \quad MDD, \quad SR.$$

This analysis would distinguish between poor performance caused by uncompensated risk and lower returns that arise naturally from the market-neutral objective.

C. Benchmarking and Ablation Studies

A central unanswered question is whether the DQN itself contributes additional value beyond the underlying statistical pairs trading framework. This can be investigated through ablation experiments.

Potential baselines include:

- 1) traditional rule-based pair selection;
- 2) CCA-V with deterministic pair selection but without DQN;
- 3) CCA-V with random pair selection;
- 4) the full CCA-V + DQN framework.

The corresponding performances should be compared using the same test period and the same trading rules. A useful comparison is

$$\Delta_{\text{DQN}} = \text{Performance}_{\text{DQN}} - \text{Performance}_{\text{baseline}}.$$

This design would provide stronger evidence regarding whether the DQN actually contributes predictive or decision-making value.

D. No-Trade Action

The current action space forces the agent to select one of the 666 candidate pairs:

$$\mathcal{A} = \{1, \dots, 666\}.$$

There is no explicit action representing “do not trade.” Therefore, even if all candidate pairs have unattractive expected rewards, the agent must still select the pair with the highest estimated Q -value.

A natural extension is

$$\mathcal{A}' = \{\text{No Trade}, 1, \dots, 666\},$$

which would allow the agent to remain inactive when the observed market state contains no sufficiently attractive trading opportunity.

E. Multi-Pair Portfolio Allocation

The current system allocates the decision to a single pair at each monthly step. A natural extension is to allow simultaneous positions in multiple pairs. The action could then be represented by a portfolio weight vector

$$\mathbf{w}_t = (w_{1,t}, \dots, w_{N,t}),$$

subject to constraints such as

$$\sum_i |w_{i,t}| \leq L$$

for a specified gross-exposure limit L .

This would reduce the concentration risk associated with selecting a single pair and would allow the model to diversify across multiple statistical arbitrage opportunities.

F. Adaptive Execution Parameters

The current trading rules use fixed parameters, including the 20-day Z-Score window and the five-day Bollinger-type threshold. Future models could allow the agent to adapt parameters such as

$$N_{\text{rolling}}, \quad \lambda_{\text{entry}}, \quad \lambda_{\text{exit}},$$

according to the observed market regime.

G. Walk-Forward Validation and Statistical Robustness

The adaptive test procedure allows the DQN to learn from previously observed test outcomes, but the evaluation should be complemented by strict walk-forward validation and multiple independent random seeds.

Performance confidence intervals, bootstrap procedures, or other statistical tests could also be used to determine whether observed differences between strategies are distinguishable from sampling variation.

VI. CONCLUSION

This study investigated a pairs trading framework that integrates residual stationarity analysis, CCA-V clustering and voting, and Deep Q-Network reinforcement learning. The statistical component constructs a candidate stock pool from residual stationarity information, while the CCA-V procedure aggregates multiple clustering configurations to identify stocks that repeatedly appear in relatively stable sub-clusters. The resulting 37-stock pool produces 666 candidate trading pairs.

The trading component represents each monthly market state using the recent Z-Score trajectories of all candidate pairs. The DQN then selects a single pair using the estimated action-value function. Importantly, the implemented temporal structure is

$$S_t \rightarrow A_t \rightarrow R_{t+1} \rightarrow S_{t+1},$$

and the model is updated only after the subsequent trading outcome has been observed.

The reported out-of-sample results show positive cumulative returns, but the strategy also exhibits modest risk-adjusted performance and substantial maximum drawdown. Therefore, the results do not support a simple conclusion that the proposed model universally outperforms conventional investment strategies. In particular, comparison with a long-only S&P 500 benchmark should be complemented by an analysis of market beta, correlation, volatility, drawdown, and risk-adjusted return.

The main value of the study is therefore not solely the absolute return achieved in the backtest. Rather, the project demonstrates the integration of statistical time-series analysis, clustering, financial signal construction, reinforcement learning, and empirical evaluation into a single quantitative research framework. The observed limitations also motivate several concrete extensions, including realistic trading costs, additional benchmark and ablation studies, historical-universe reconstruction, no-trade decisions, multi-pair portfolio allocation, and more rigorous walk-forward validation.

References

- [1] H.-Y. Lai. Pairs trading based on cointegration clustering and reinforcement learning. Master's thesis, Graduate Institute of Statistics, National Central University, Taiwan, 2025.
- [2] Y. Xia, T. Wang, S. C. Liew, and S. Zhang. A hierarchical deep learning framework for pairs trading with attention and graph networks. *Expert Systems with Applications*, 2025.
- [3] X. Xu and Chao Lan. Improved pairs trading strategy using two-level reinforcement learning framework. *Engineering Applications of Artificial Intelligence*, 2023.